ЛАБОРАТОРНАЯ РАБОТА №3

Тестирование чисел на простоту и построение больших простых чисел

Цель работы

Освоить основные программные методы тестирования чисел на простоту

3.1. Метод пробных делений

Если n – составное, то $n = a \cdot b$, где $1 < a \le b$ причем $a \le \sqrt{n}$. Поэтому для $d = \overline{2, \lfloor \sqrt{n} \rfloor}$ мы проверяем, делится ли n на d? Если делитель числа n не будет найден, то n – простое. В противном случае будет найден минимальный простой делитель числа n, т.е. n будет разложено на два множителя. Сложность метода составляет $O(\sqrt{n})$ арифметических операций с целыми числами.

Возможны модификации этого метода. Например, мы можем проверить, делится ли n на *2* и на *3* , и если нет, то перебираем далее только числа d вида $1 + 6j, 5 + 6j,\ j = 1, 2, \ldots$.

3.2. Решето Эратосфена

Если мы хотим составить таблицу всех простых чисел среди чисел $2, 3, \ldots, n$ то нужно сначала вычеркнуть все числа, делящиеся на *2*, кроме *2*. Затем взять число *3* и вычеркнуть все последующие числа, делящиеся на *3* . Затем взять следующее не вычеркнутое число (т. е. *5*) и вычеркнуть все

последующие делящиеся на него числа, и так далее. В итоге останутся лишь простые числа.

Для реализации метода нужен большой объем памяти ЭВМ, однако для составления таблиц простых чисел он является наилучшим.

Всего требуется $O(\log^3 n)$ арифметических операций.

3.3. Критерий Вильсона

Теорема 1[1].

Для любого n следующие условия эквивалентны:

1. n – простое;
2. $(n-1)! \equiv -1 (\mathrm{mod}\ n)$

Данный критерий иногда бывает удобен в доказательствах, но применять его для проверки простоты невозможно ввиду большой трудоемкости.

3.4. Тест на основе малой теоремы Ферма

Малая теорема Ферма утверждает, что если n – простое, то выполняется условие: $\forall a = \overline{2, n-1}$ имеет место сравнение:

$$a^{n-1} \equiv 1 (\mathrm{mod}\ n) \qquad \textbf{(1)}$$

Обратное утверждение неверно.

Из этой теоремы следует, что если (1) не выполнилось хотя бы для одного числа a в интервале $[2; n-1]$, то n – составное. Поэтому можно предложить следующий вероятностный тест простоты:

1. Выбираем случайное число из интервала $[1; n-1]$ и проверяем с помощью алгоритма Евклида[2] условие $НОД(a, n) = 1$.
2. Проверяем выполнимость сравнения $a^{n-1} \equiv 1 (\mathrm{mod}\ n)$.
3. Если сравнение (1) не выполнено, то ответ n – составное

---

[1] В 1770 г. Э. Варинг опубликовал эту теорему, приписываемую Д. Вильсону.

[2] См. прил. 1.

4. Если сравнение (1) выполнено то ответ неизвестен но можно повторить тест еще раз.

Если выполняется сравнение (1), то говорят, что число n является псевдопростым по основанию a. Заметим, что существует бесконечно много пар чисел (a, n), где n – составное и псевдопростое по основанию a. Например, при $(a, n) = (2, 341)$ получаем $2^{340} = \left(2^{10}\right)^{34} \equiv 1 (\mathrm{mod}\ 341)$, хотя $341 = 11 \cdot 31$.

Особый случай составляют составные числа, для которых условие сравнения выполняется при всех основаниях. Они называются псевдопростыми числами, или числами Кармайкла.

Таким образом, при применении описанного выше теста может возникнуть три ситуации:

- число n простое и тест всегда говорит «не известно»;
- число n составное и не является числом Кармайкла; тогда с вероятностью успеха не меньше $\frac{1}{2}$ тест дает ответ «n – составное»;
- число n составное и является числом Кармайкла, тогда тест всегда дает ответ «не известно».

Числа Кармайкла являются достаточно редкими. Так, имеется всего *2163* чисел Кармайкла не превосходящих $25 \cdot 10^9$. До 10^5 числами Кармайкла являются только следующие *16* чисел: *561, 1105, 1729, 2465, 2821, 6601, 8911, 10585, 15841, 29341, 41041, 46657, 52633, 62745, 63973* и *75361*. Проверка того, является ли заданное число числом Кармайкла, требует нахождения разложения числа на простые сомножители, т. е. факторизации числа. Поскольку задача факторизации чисел является более сложной, чем задача проверки простоты, то предварительная отбраковка чисел Кармайкла не представляется возможной. Поэтому в приведенном выше тесте простые числа и числа Кармайкла полностью неразличимы.

6.5. Тест Соловея – Штрассена

Теорема 2.

Для любого нечетного n следующие условия эквивалентны:

1. n – простое;
2. $\forall a$:

$$a^{\frac{n-1}{2}} \equiv \left(\frac{a}{n}\right) (\mathrm{mod}\ n) \qquad \textbf{(2)}$$

где $\left(\frac{a}{n}\right)$ – символ Лежандра.

Р. Соловей и В.Штрассен предложили следующий вероятностный тест для проверки простоты чисел:

1. Выбираем случайное число a из интервала $[1;\ n-1]$ и проверяем с помощью алгоритма Евклида условие $НОД(a, n) = 1$.
2. Если оно не выполняется, то n – составное.
3. Проверяем выполнимость сравнения (2).
4. Если оно не выполняется, то n – составное.
5. Если сравнение выполнено, то ответ неизвестен (и тест можно повторить еще раз).

Сложность данного теста, как и теста на основе малой теоремы Ферма, оценивается величиной $O(\log^3 n)$.

Данный тест полностью аналогичен тесту на основе малой теоремы Ферма, однако, он обладает решающим преимуществом – при его использовании возникает только две ситуации:

- число n простое и тест всегда говорит «не известно»;
- число n составное и тест с вероятностью успеха не меньше $\frac{1}{2}$ дает ответ «n – составное».

После повторения теста k раз вероятность неотбраковки составного числа не превосходит $\frac{1}{2^k}$.

3.6. Тест Леманна.

Далее приведена последовательность действий при проверке простоты числа n:

1. Выбрать случайно число a, меньшее n.
2. Вычислить $t = a^{\frac{p-1}{2}} \bmod n$.
3. Если $t \neq \pm 1$ (или или), то n не является простым.
4. Если $t \equiv \pm 1$ (или или), то вероятность того, что число n не является простым, не больше $\frac{1}{2}$.

Повторите эту проверку k раз. Если результат вычислений равен 1 или – 1, но не всегда равен 1, то n является простым числом с вероятностью ошибки $\frac{1}{2^k}$.

3.7. Тест Рабина – Миллера[4]

Пусть n – нечетное и $n - 1 = 2^s \cdot t$, t – нечетное. Если число n является простым, то при всех $a \geq 2$ выполняется сравнение $a^{n-1} \equiv 1 (\bmod n)$. Поэтому, рассматривая элементы $a^t, a^{2t}, \ldots, a^{2^{s-1}t}$ можно заметить, что либо среди них найдется равный $-1 (\bmod n)$, либо $a^t \equiv 1 (\bmod n)$.

На этом замечании основан следующий вероятностный тест простоты:

1. Выбираем случайное число a из интервала $[1; n - 1]$ и проверяем с помощью алгоритма Евклида условие $НОД(a, n) = 1$.
2. Если оно не выполняется, то ответ «n – составное».
3. Вычисляем $a^t \bmod n$.

---

4 Предложен М. Рабином в 1980 г.

4. Если $a^t \equiv \pm 1 (\mathrm{mod}\ n)$, то переходим к шагу 1.

5. Вычисляем $a^t \bmod n, a^{2t} \bmod n, \ldots, a^{2^{s-1}t} \bmod n$ до тех пор, пока не появится -1.

6. Если ни одно из этих чисел не равно -1, то ответ «n – составное»;

7. Если мы достигли n, то ответ неизвестен (и тест можно повторить еще раз).

Арифметическая сложность данного теста, очевидно, составляет $O(s \cdot n)$.

После повторения данного теста k раз вероятность неотбраковки составного числа не превосходит $\frac{1}{4^k}$.

Тест Рабина – Миллера всегда сильнее теста Соловея – Штрассена. Точнее, если при фиксированном n число a проходит тест Рабина – Миллера и не показывает, что n составное, то оно проходит тест Соловея – Штрассена с тем же результатом.

3.8. Полиномиальный тест распознавания простоты.

Данный алгоритм основан на следующем критерии простоты.

Теорема 3.

Пусть числа a, n взаимно просты. Тогда n – простое в том и только в том случае, когда выполнено сравнение:

$$(x - a)^n \equiv (x^n - a) (\mathrm{mod}\ n) \qquad \textbf{(3)}$$

Приведем сам алгоритм.

Вход: целое $n > 1$.

1. Если число n имеет вид a^b, то ответ «n – составное».
2. $r \leftarrow 2$
3. Цикл пока $r < n$:
4. Если $НОД(r, n) \neq 1$, то ответ «n – составное».

5. Если r – простое, то вычислить q – наибольший простой делитель $r - 1$; если $\left(q > 4\sqrt{r}\ \log_2 n\right)$ и $\left(n^{\frac{r-1}{q}} \not\equiv 1 \pmod r\right)$, то выйти из цикла.
6. $r \leftarrow r + 1$.
7. Завершить цикл.
8. Если $r = n$, то ответ «n – составное».
9. Если $n - 1 \le \left[2\sqrt{r}\ \log_2 n\right]$, то $\forall a = \overline{r, n-1}$ проверить выполнение условия: $НОД(a, n) = 1$.
10. Если $n - 1 > \left[2\sqrt{r}\ \log_2 n\right]$, то $a = \overline{1, 2\sqrt{r}\ \log_2 n}$ проверить выполнение условия: $(x - a)^n \equiv \left(x^n - a\right)\left(\mathrm{mod}\ x^r - 1\right)$.
11. Если на шаге 9-10 нарушились равенства, то ответ «n – составное».
12. Если мы дошли до этого шага, то ответ «n – простое».

Задание

1. Реализовать приложение, позволяющее генерировать простое число по следующей схеме (с использованием, например, теста Рабина – Миллера):

    1. Сгенерируйте случайное p-битовое число n.
    2. Установите старший и младший биты равными *1*. (Старший бит гарантирует требуемую длину простого числа, а младший бит обеспечивает его нечетность.)
    3. Убедитесь, что n не делится на небольшие простые числа: 3, 5, 7, 11, и т.д. Во многих реализациях проверяется делимость n на все простые числа, меньшие 256. Наиболее эффективной является проверка на делимость для всех простых чисел, меньших 2000.

    Все простые числа, не превосходящие *256* перечислены ниже:
    2,3,5,7,11,13,17,19,23,29,31,37,41,43,47,53,59,61,67,71,73,79,
    83,89,97,101,103,107,109,113,127,131,137,139,149,151,157,
    163,167,173,179,181,191,193,197,199,211,223,227,229,233,239,
    241,251.

4. Выполните один тест Рабина – Миллера проверки простоты числа для некоторого случайного a. Если n проходит тест, сгенерируйте другое случайное a и повторите проверку. Выбирайте небольшие значения a для ускорения вычислений. Выполните пять тестов. (Одного может показаться достаточным, но выполните пять.) Если n не проходит одной из проверок, сгенерируйте другое n и попробуйте снова.

Можно не генерировать n случайным образом каждый раз, но последовательно перебирать числа, начиная со случайно выбранного до тех пор, пока не будет найдено простое число.

Вместо теста Рабина – Миллера следует использовать тест, заданный в варианте.

Вариант	Алгоритм
1	Тест Соловея – Штрассена
2	Тест Леманна.
3	Критерий Вильсона+Решето Эратосфена
4	Тест Рабина – Миллера
5	Тест на основе малой теоремы Ферма
6	Полиномиальный тест распознавания простоты.

Контрольные вопросы

1. Простые числа.
2. Метод пробных делений
3. Решето Эратосфена
4. Критерий Вильсона
5. Тест на основе малой теоремы Ферма

Приложение 1

Код функции, реализующий алгоритм Евклида

```
/* возвращает НОД (gcd) x и y */
int gcd (int x, int y) {
int g;
if (x < 0)
x = -x;
if (У < 0)
y = -y;
if (x + y == 0 )
ERROR ;
g = y;
while   (x > 0)
{
g = x; x = y % x;
y = g;
}
```